Rubberband Baseline Correction with Local Bending of Concave Regions

Thanya Manivannan, 300686753
School of Mathematics and Statistics, Victoria University of Wellington
thanyamanivannan@gmail.com

***Abstract*—The rubberband method estimates the baseline of a Fourier transform infrared (FTIR) spectrum as the lower convex hull of the data. A lower convex hull cannot follow a baseline that bends downwards, so a concave baseline remains in the corrected spectrum. This report describes hullBase, an R package that keeps the plain rubberband baseline where it fits, flags the hull segments that hide a concave region, and bends only those segments. The location of the bending is decided by a threshold calculated from the noise level of the spectrum, the strength of the bending is calculated from the estimated height of the concave region, and the peak width is set by the user. The package has one main function, an S3 result class, two C++ routines and 452 passing test expectations. On 300 simulated spectra with known baselines, the mean baseline error on concave baselines fell from 0.745 to 0.056 at 30 dB, and flat, linear and convex baselines were unchanged. The report also describes the development process, checks the package against published reproducibility criteria and states the limitations of this package.**

***Index Terms*—Baseline correction, convex hull, FTIR spectroscopy, R package, concave region, rubberband method.**

I. INTRODUCTION

An Fourier Transform Infrared spectrometer measures the amount of infrared light that a sample absorbs at each wavenumber. Usually the wavenumber (in cm^{-1}) is plotted on the horizontal x axis of the spectrum, while the absorbance is plotted on the vertical y axis. The sample chemical information is carried by the peaks. The baseline, which is produced by the instrument, light scattering is a gradually shifting background beneath the peaks. The peak heights and areas can only be compared once the baseline has been removed.

The rubberband method estimates the baseline as the lower convex hull of the points (x_i, y_i), and the corrected spectrum is subtracting the estimated baseline from measured signal or intensity $(y - b)$. Rubberband method stretch a rubberband underneath the low lying points of spectrum called hull points, and the straight lines that connect each hull point with its neighbouring hull points are called hull segments. Between two points that the hull touches, the hull is a straight line, and at each touching point it can only bend upwards. When the true baseline bends downwards in a region, a concave region, the hull touches only the two ends of the region and runs straight underneath it. The gap between this line and the true baseline remains in the corrected spectrum, and peaks on that region are measured too tall.

A manual correction for this limitation was described in [1]. In this approach, a quadratic bending curve that bends upwards is added to the whole spectrum. The upward bend of the curve cancels the downward bend of the concave region, so the concave region becomes convex and the lower convex hull can follow it. The hull is taken on this bent spectrum, and the same curve is then subtracted from the hull to give the baseline. In [1], the bending strength, which sets how far the spectrum is bent, is chosen by eye, and one bending strength is applied to the whole spectrum. The R package hullBase uses the same bend, hull and subtract steps, but applies them only where they are needed and calculates the bending strength from the data. First, the residual above each hull segment is eroded with a moving minimum as wide as the user given peak width. The erosion removes the peaks, and the eroded residual that remains is an estimate of the concave region. A hull segment is flagged when its eroded residual exceeds a threshold calculated from the noise level of the spectrum. Second, the bending curve is added only to the flagged segment and is zero at both ends of that segment, so every other hull segment keeps the plain rubberband line. Third, the bending strength is set to twice the estimated height of the concave region, which is taken from the eroded residual. The hullBase baseline b therefore follows the concave region without the true baseline being known, and the corrected spectrum is $y - b$.

The project addressed one limitation of the rubberband method, namely that the lower convex hull cannot follow a concave region of the baseline. Flat, linear and convex baselines are already followed by the plain rubberband baseline, so hullBase keeps the rubberband method as the primary method and adds the detection and correction of concave regions. The objectives were to

1) compute the plain rubberband baseline as the lower convex hull, with the hull loop written in C++,
2) detect the hull segments that hide a concave region, using erosion and a threshold calculated from the noise level,
3) apply local bending only to the flagged hull segments and keep the plain rubberband line on every other segment,
4) return one result object of class BaselineModel with print(), summary() and plot() methods,
5) evaluate the package with unit tests and with simulated spectra whose true baseline is known.

II. HULLBASE PACKAGE

A. Functions and Interface

Table I lists the functions of hullBase. The user calls one main function, correct(), which runs the full baseline correction and returns the result. Each step of the correction is also available as a separate function, so every stage can be run, inspected and tested on its own. The function correct() is an S3 generic with two methods, a default method for two numeric vectors x and y, and a method for a data frame with columns named x and y. Every function that takes a spectrum as input first calls the internal function check_spectrum(), which checks the input, sorts the points by wavenumber and stops or warns with its own message for each problem found.

TABLE I
MAIN FUNCTIONS OF HULLBASE

Function	What it does in hullBase
correct()	runs the full correction and returns a BaselineModel
lower_hull()	returns the plain rubberband baseline

Function	What it does in hullBase
noise_estimate()	returns the noise level of the spectrum
detect_concave_segments()	lists every hull segment and flags the ones that hide a concave region
refine_segment()	bends one flagged segment and returns its new baseline
simulate_spectrum()	creates a test spectrum with a known true baseline
print(), summary(), plot()	show, summarise and plot a BaselineModel
lower_hull_cpp(), erode_cpp()	internal C++ functions for the hull and the moving minimum
check_spectrum(), bend_segment()	internal R functions for the input checks and the bending steps

The function correct() returns a list of class BaselineModel with ten fields, namely the input x and y, the hullBase baseline, the corrected spectrum, the plain rubberband baseline, the noise level, peak_width, method, a segments table that records the decision for every hull segment, and the call. The plain rubberband baseline is stored in the object so that every hullBase result can be compared with it. The method print() gives a short overview of the correction, and summary() reports the noise level, the threshold, the number of hull segments, the refined segments and how far the hullBase baseline was lifted above the plain rubberband baseline. The method plot() has two views, one that shows the spectrum with both baselines and shades the refined segments, and one that shows the corrected spectrum. Fig. 1 shows how data moves through the package.

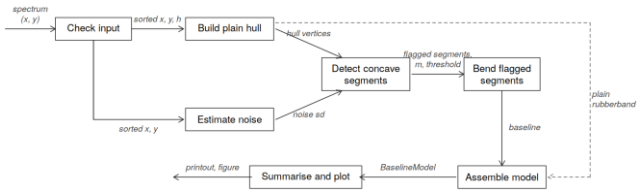


Fig. 1. How the correct() function works.

B. How correct() function works

One simulated spectrum is used as an example to show how the function correct() works step by step. The spectrum was created with simulate_spectrum() using its default settings, which is the same design as the simulated spectra used in the results. It has 1401 points from 0 to 1400 in steps of 1, eight Gaussian peaks, a concave baseline of height 1 and noise at 30 dB. The range from 0 to 1400 holds all eight peaks with space at both ends. The peak width of 300 was chosen to cover the widest group of overlapping peaks. The spectrum was created and corrected with the following code.

```
library(hullBase)
set.seed(1)
sim <- simulate_spectrum(baseline = "concave",
                         snr_db = 30)
fit <- correct(sim, peak_width = 300)
summary(fit)
```

The function correct() uses only x, y and the peak width, and the true baseline and noise stored by simulate_spectrum() are used only to check the result. Table II lists the values produced inside correct() and the checks against the true values. Only the segment from 13 to 1396, which lies under the concave region, was flagged, and on this segment the RMSE fell from 0.742 to 0.054. The accuracy of the method in general is tested on 300 simulated spectra in the results section.

TABLE II
VALUES FOR THE SIMULATED SPECTRUM IN THE WORKED CASE

Quantity	Value
Produced inside correct()	
noise estimate	0.0335
moving minimum window half width m (points)	150
detection threshold	0.255
hull segments, refined segments	5, 1
largest eroded residual on the refined segment	0.936
estimated height, bending strength	1.033, 2.066
largest lift above the plain rubberband	0.972
Checked against the true values	
true noise	0.0322
RMSE on the refined segment, plain rubberband / hullBase	0.742 / 0.054
baseline outside the refined segment equal to the plain rubberband	yes

III. METHOD AND IMPLEMENTATION

A. Lower convex hull

This is the first step in hullBase. In this the plain rubberband baseline is computed by the function lower_hull() and the same step is run inside correct(). The function first calls check_spectrum(), which sorts the points by increasing wavenumber. The sorted points are then passed to the internal C++ function lower_hull_cpp(), which finds the lower hull points of the spectrum with Andrew monotone chain algorithm [2] and because the points are sorted, lower_hull_cpp() finds the hull points in one scan from left to right. For each new point P, the last two kept points O and A are checked, and the direction of the turn from O through A to P is given by the two dimensional cross product

$$\begin{aligned}\mathrm{cross}(O,A,P) = (x_A - x_O)(y_P - y_O) \\ - (y_A - y_O)(x_P - x_O)\end{aligned} \quad (1)$$

When cross ≤ 0, A lies on or above the line from O to P and is removed, and the check is repeated before P is kept. The function lower_hull() joins the returned hull points with straight lines, puts the baseline back in the original order, and correct() stores it in the rubberband field of the BaselineModel. The R copy lower_hull_r() is kept so the tests can confirm it finds the same hull points as the C++ version. For y = 5, 2, 3, 2, 5 at x = 1 to 5, the point with y = 3 lies above the line between its neighbours and is removed, so lower_hull() returns 5, 2, 2, 2, 5.

B. Noise Estimate

The detection of concave segments depends on the noise level of the spectrum since the detection threshold is calculated from it. In the package hullbase, the noise standard deviation is estimated by the function noise_estimate(), which is called by correct() and detect_concave_segments() when the user does not give a noise value. The estimate is calculated from the second differences of the sorted spectrum,

$$\hat{\sigma} = \frac{\mathrm{MAD}(y_{i+1} - 2y_i + y_{i-1})}{\sqrt{6}} \quad (2)$$

In this, a second difference is zero on a straight line and nearly constant on a smooth baseline, so what changes from point to point is mainly the noise. Also, the median absolute deviation (MAD) [3] measures the spread of these values and is not inflated by the few large values at peaks. The MAD includes the usual factor 1.4826, so that it equals the standard deviation for Gaussian noise. For independent noise the second difference has variance $6\sigma^2$, which gives the factor $\sqrt{6}$. The estimate is stored in the noise field of the BaselineModel. For the example spectrum the estimate was 0.0335 against a true value of 0.0322.

C. Detect concave segments

In the package hullBase, hull segments that hide a concave region are found by the function detect_concave_segments(). Firstly, for each hull segment, the residual $r_i = y_i - b_i$ is calculated as the height of the spectrum above the straight hull segment. This residual contains the peaks and when a concave region lies under the segment it also contains the height of that concave region above the straight hull segment. Second, the residual is eroded with a moving minimum [4] and a window of 2m + 1 points slides along the residual and keeps the smallest value at each point. Since the window is wider than any peak, the peaks are removed and only the wider concave region remains,

$$e_i = \min_{|j-i| \le m} r_j\,, \quad m = \mathrm{round}\left(\frac{w}{2h}\right) \quad (3)$$

w indicates peak_width, the width of the widest peak and h is the step between neighbouring wavenumbers. And erosion is run by the C++ function erode_cpp(), which uses a monotone queue [5] to keep track of smallest value in the window, so the time it takes does not grow with the window size. Thirdly, the largest eroded value in the segment is compared with a threshold set from the noise level $\hat{\sigma}$, and the segment is only flagged when

$$\max_i e_i > 2\hat{\sigma}\sqrt{2\ln n} \quad (4)$$

The value $\hat{\sigma}\sqrt{(2 \ln n)}$ is the universal threshold [6], which is about the largest value that the noise alone is expected to reach in a spectrum of n points. The threshold in hullBase is twice this value so a segment is flagged only when its eroded value is clearly higher than the noise. With segments fewer than 2m + 3 points are not tested because they are too short to hide a region wider than a peak. For every hull segment, the largest eroded value and whether the segment was flagged are stored in the segments table of the BaselineModel, so the user can see why each segment was or was not refined.

D. Local Bending of a Flagged Segment

In package hullBase, each flagged segment is bent by the internal function bend_segment(). The user can also bend a single segment with refine_segment(). The segment lies between the hull points x_L and x_R and has a width of $W = x_R - x_L$. First, a convex quadratic bending curve is defined,

$$q(x) = \frac{(x - x_L)(x - x_R)}{(W/2)^2} \quad (5)$$

which is zero at both ends of the segment and −1 at its centre. Since the curve is convex by adding it to the spectrum cancels the curvature of the concave region. Secondly the bending strength is calculated from the eroded residual e of (3) which shows the height of the concave region above the straight hull segment. Here, the ratio of e to the curve over the central half of the segment gives the estimated height of the concave region at its centre, and the bending strength c is k times this height,

$$c = k \max_{q(x) \le -1/2} \frac{e(x)}{-q(x)} \quad (6)$$

where k is bend_factor, which is 2 by default. The factor gives a margin, because a concave region with a half sine profile which is the profile of the simulated concave baseline, needs about $\pi^2/8 \approx 1.23$ times its height to be fully bent. Thirdly the bending curve is added to the spectrum with strength c, the lower convex hull of the bent spectrum is calculated, and the curve is subtracted again,

$$b = \min\{\mathrm{hull}(y + cq) - cq, y\} \quad (7)$$

so that the baseline b follows the concave region. By taking the minimum with y keeps the baseline below the spectrum. And finally, if part of the concave region still exceeds the threshold after one bend, bend_segment() repeats these steps on the remaining part, up to max_depth = 3 levels. For a noise free half sine profile of height 2 with k = 1, the largest remaining error decreased from 0.115 after one level to 0.029 after two and 0.007 after three.

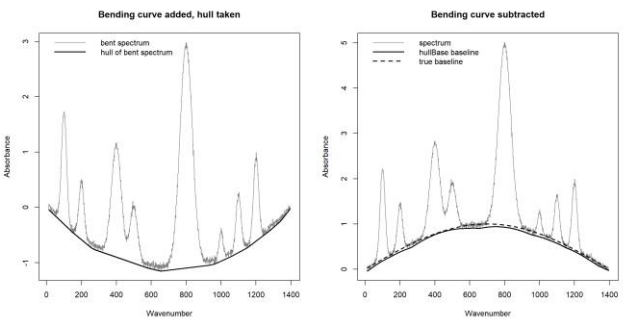


Fig. 2. Local bending on the simulated spectrum. Left shows the bending curve makes the concave region convex and the hull follows the bent spectrum. Right shows that after the curve is subtracted, the hullBase baseline follows the true baseline.

IV. TESTING

The package hullBase is tested with testthat [7], with one test file for each part of the package. The tests consist of 79 test_that() blocks, each testing one behaviour, and 452 expectations, each a single check of one result. There are more expectations than blocks because several blocks repeat the same check on many randomly generated spectra, so that a rule is confirmed across many inputs and not only for a single test

case. Table III shows how the expectations are distributed across the test files and why each file has its number.

TABLE III
NUMBER OF EXPECTATIONS IN EACH TEST FILE

Test file	Blocks	Expectations	Why this number
check_spectrum	10	13	one check per input rule, plus 3 for NA, NaN and Inf
compiled	3	104	one block repeats its check on 100 random inputs
correct	16	40	many small checks of the result object and its methods, 1 to 5 per block
detect_concave_segments	12	80	one block repeats its check on 60 simulated spectra (3 shapes × 2 noise levels × 10)
lower_hull	9	163	three blocks each repeat their check on 50 random spectra
noise_estimate	8	12	one check per block, plus 5 for the five baseline shapes
refine_segment	10	18	one or a few checks per block
simulate_spectrum	11	22	one or a few checks per block
Total	**79**	**452**	

devtools::test() gave 452 passes with 0 failures, 0 warnings and 0 skips, and devtools::check() gave 0 errors, 0 warnings and 0 notes. The package was also checked on a second platform (Linux, R 4.3.3), where all 452 expectations passed. These results show that each function behaves as written. They do not show that the hullBase baseline is accurate, which is tested on simulated spectra in the results section.

V. RESULTS

A. Baseline Correction on the Simulated Spectra Dataset

The simulated spectra dataset contains 300 spectra from simulate_spectrum() [8], with eight Gaussian peaks, Gaussian noise and five baseline shapes, 20 spectra for each shape at 30, 25 and 20 dB, corrected with peak_width = 300 and set.seed(2026). Table IV gives the mean RMSE between each estimated baseline and the true baseline. The error fell only on concave and mixed baselines, while flat, linear and convex baselines were not flagged and stayed the same as the plain rubberband (Fig. 3). The correction failed when the concave region was lower than about 7.6 times the noise level, and when the noise at 10 dB raised the threshold above the concave baseline.

TABLE IV
MEAN BASELINE RMSE ON THE SIMULATED SPECTRA DATASET, PLAIN RUBBERBAND / HULLBASE

Baseline shape	30 dB	25 dB	20 dB
concave	0.745 / 0.056	0.796 / 0.114	0.895 / 0.223
mixed	0.787 / 0.069	0.823 / 0.105	0.904 / 0.204
flat	0.089 / 0.089	0.153 / 0.153	0.270 / 0.270
linear	0.086 / 0.086	0.151 / 0.151	0.274 / 0.274
convex	0.057 / 0.057	0.112 / 0.112	0.227 / 0.227

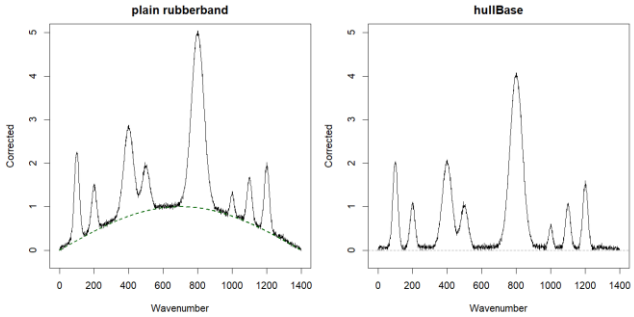


Fig. 3. One simulated spectrum with a concave baseline after correction. Left, after the plain rubberband the concave baseline remains. Right, after hullBase the corrected spectrum lies at zero between the peaks.

B. Baseline Correction on the Simulated FTIR Spectra Dataset

The simulated FTIR spectra dataset has ten Gaussian peaks at typical FTIR positions from 400 to 4000 cm^{-1} on a concave baseline of height 0.3 at 30 dB. Table V shows that over 20 spectra hullBase reduced the baseline error and brought the height of the 2920 cm^{-1} peak close to its true value (Fig. 4). The correction failed when peak_width was smaller than the widest peak, and with peak_width = 50 the broad peak at 3300 cm^{-1}, with a true height of 0.25, was lifted into the baseline and measured as 0.026.

TABLE V
RESULTS ON THE SIMULATED FTIR SPECTRA DATASET, MEAN OVER 20 SPECTRA

Measure	True value	Plain rubberband	hullBase
baseline RMSE, concave	0	0.217	0.010
height of the 2920 cm^{-1} peak	0.80	1.051	0.812

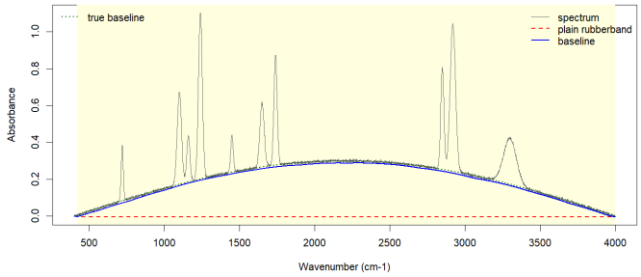


Fig. 4. Simulated FTIR spectrum on a concave baseline. The plain rubberband (dashed) lies far below the true baseline (dotted), while the hullBase baseline (solid) follows it across the refined segment (shaded).

C. Baseline Correction on the SrE.IR Dataset

The SrE.IR dataset from the ChemoSpec package [9] contains 16 measured FTIR spectra with no known true baseline, so the results were checked for plausibility. Table VI shows that the corrected spectrum was never negative at either peak width, and that fewer segments were refined at peak_width = 800 (Fig. 5). The baseline under the band group between 500 and 1500 cm^{-1} changed with peak_width, and at peak_width = 300 the band group was lifted into the baseline.

TABLE VI
RESULTS ON THE SRE.IR DATASET, RANGE OVER 16 SPECTRA

peak_width	Segments refined	Largest lift above plain rubberband	Lowest corrected value
300	2 to 3	0.0316 to 0.0465	0
800	0 to 1	0 to 0.0202	0

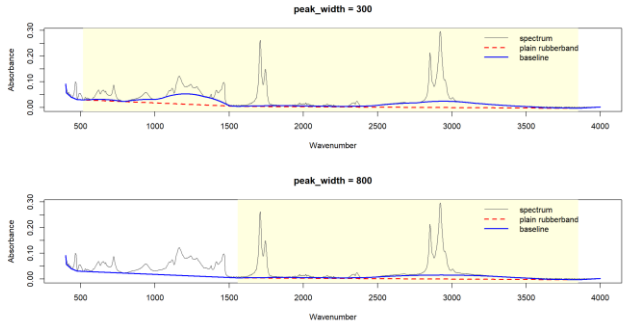


Fig. 5. Measured FTIR spectrum, SrE.IR sample 1. Top, with peak_width = 300 the band group between 500 and 1500 cm^{-1} is lifted into the baseline. Bottom, with peak_width = 800 it is kept.

VI. REFLECTION AND FUTURE WORK

A. Development and Reflection

The package was developed one function at a time, with each function tested before the next was written. Several first approaches were replaced during development. The first noise estimate used first differences, which counted the slope of a curved baseline as noise, and was replaced by second differences. The first detection rule searched for local dips in the residual, but a valley between two peaks also forms a dip, so it was replaced by erosion with a noise threshold. The first refinement replaced each point by the mean of its neighbours, which needed hundreds of passes on a wide segment, and was replaced by local bending. Writing the peer assessment plan showed that correct() did not check noise, bend_factor or max_depth, and these checks were added and tested. The peer assessment of the package did not lead to a change in the code. If the project were started again, the simulator and the evaluation would be built first, so that every method could be measured against a known baseline from the start.

B. Future Work

Table VII lists the changes planned for future versions of hullBase, each linked to a limitation found in this report.

TABLE VII
PLANNED CHANGES TO HULLBASE

Limitation	Planned change	Why
peak_width is chosen by the user	estimate the peak width from the spectrum	the result changed with peak_width on the SrE.IR and simulated FTIR datasets
concave regions lower than about 7.6 times the noise level are not detected	test other threshold factors	the factor of two was fixed and other values were not tested
Lorentzian peaks were not evaluated	add Lorentzian peaks to simulate_spectrum() and repeat the evaluation	only Gaussian peaks were used in the main evaluation
correlated noise	a noise estimate that allows for correlated noise	the factor √6 assumes independent noise, and interpolation makes the noise correlated
noise that changes across the spectrum	estimate the noise level for each segment	one noise level is used for the whole spectrum
uneven spacing must be interpolated	set the erosion window in wavenumber units	interpolation changes the noise
transmittance spectra are not converted	an argument that converts transmittance to absorbance	the rubberband needs peaks that point upwards
the baseline is made of straight pieces	an optional smooth curve through the hull points	the baseline has corners at the hull points
no comparison with other baseline methods	compare with other methods such as [8]	the relative performance of hullBase is unknown

VII. DOCUMENTATION AND REPRODUCIBILITY

The package is installed from GitHub with install.packages("pak") then pak::pak("ThanyaManivannan/hullBase"). The README and vignette describe how to install and use the package, and each exported function has a help page with a working example. Table VIII compares hullBase with the reproducibility requirements of [10].

TABLE VIII
HULLBASE AGAINST THE REPRODUCIBILITY REQUIREMENTS OF [10]

Requirement	Status	Evidence
Public code	Met	GitHub, MIT licence, help page for every function
Free software	Met	R, Rcpp, testthat, knitr, rmarkdown
Install and run instructions	Met	README and vignette
Data available	Met	simulate_spectrum(), SrE.IR from a CRAN package
Seeds given	Met	set.seed(1), set.seed(2026), set.seed(101)
Platform and versions	Partly	Windows, R 4.5.2, no full session record
Scripts for tables and figures	Partly	kept outside the repository
Permanent identifier	Not met	git tag v0.1.0 only, no DOI

VIII. CONCLUSION

The R package hullBase keeps the rubberband method as the primary method for FTIR baseline correction and adds the detection and correction of concave regions. The package flags the hull segments that hide a concave region using erosion and a threshold calculated from the noise level, and bends only those segments with a bending strength calculated from the estimated height of the concave region. The package provides one main function, correct(), a BaselineModel result object with print(), summary() and plot() methods, two C++ functions and 452 passing test expectations. On the simulated spectra, the baseline error fell on concave and mixed baselines and was unchanged on flat, linear and convex baselines, and on the measured spectra the corrected spectra were never negative. The correction depends on the peak width chosen by the user, does not detect concave regions lower than about 7.6 times the noise level on the simulated spectra, and was tested mainly on Gaussian peaks with independent noise. These results do not

show that the method is accurate on all measured spectra or better than other baseline methods.

STATEMENT

This report and the hullBase package were developed by the author. The package was written in R with C++ through Rcpp, documented with roxygen2, tested with testthat and checked with devtools. GitHub and Copilot was used as a coding assistant while writing the package code. All results come from the package, and all content was reviewed and approved by the author. The package is available at https://github.com/ThanyaManivannan/hullBase as version 0.1.0 with git tag v0.1.0 under the MIT licence.

REFERENCES

[1] C. Beleites, "Fitting baselines to spectra," vignette of the R package hyperSpec, 2015.

[2] A. M. Andrew, "Another efficient algorithm for convex hulls in two dimensions," *Inf. Process. Lett.*, vol. 9, no. 5, pp. 216–219, 1979.

[3] P. J. Rousseeuw and C. Croux, "Alternatives to the median absolute deviation," *J. Amer. Statist. Assoc.*, vol. 88, no. 424, pp. 1273–1283, 1993.

[4] J. Serra, *Image Analysis and Mathematical Morphology*. London, U.K.: Academic Press, 1982.

[5] D. Lemire, "Streaming maximum-minimum filter using no more than three comparisons per element," *Nordic J. Comput.*, vol. 13, no. 4, pp. 328–339, 2006.

[6] D. L. Donoho and I. M. Johnstone, "Ideal spatial adaptation by wavelet shrinkage," *Biometrika*, vol. 81, no. 3, pp. 425–455, 1994.

[7] H. Wickham, "testthat: Get started with testing," *The R Journal*, vol. 3, no. 1, pp. 5–10, 2011.

[8] F. Zhang, X. Tang, A. Tong, B. Wang, and J. Wang, "An automatic baseline correction method based on the penalized least squares method," *Sensors*, vol. 20, no. 7, Art. no. 2015, 2020.

[9] B. A. Hanson, "ChemoSpec: Exploratory chemometrics for spectroscopy," R package, CRAN. [Online]. Available: https://CRAN.R-project.org/package=ChemoSpec

[10] A. C. Frery, L. Gomez, and A. C. Medeiros, "A badging system for reproducibility and replicability in remote sensing research," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 4988–4995, 2020, doi: 10.1109/JSTARS.2020.3019418.